

ROS 2 URDF of 4-DOF Industrial Arm

URDF Modeling of a 4-DOF Industrial Robotic Arm in ROS 2

Project Report
Industrial Training Program on Robotics and AI

Submitted by: Dhruv Khanna

Date: 20 July 2026

Environment note: This report, the URDF/xacro model, and the launch files were built and statically validated (well-formed XML, single-root kinematic tree, joint limit sanity checks) in an environment without ROS 2 / RViz2 installed. The images in Sections 5 and 6 below are geometry-accurate schematics generated directly from this URDF's own joint origins/axes, used as a stand-in for real RViz/TF-tree screenshots. Section 8 gives the exact commands to run on a real ROS 2 (Humble/Iron) workstation to capture the actual RViz and view_frames screenshots required by the submission guidelines.

1. Problem Statement

Design and implement a ROS 2 URDF model of a 4-DOF industrial robotic arm that defines all links and joints correctly, includes visual and collision elements, uses proper joint types and limits, loads successfully in RViz, and displays a correct TF tree.

2. Robot Structure Design (Question 1)

The arm is modelled as a serial kinematic chain with 4 revolute joints (the required 4 degrees of freedom) plus 2 fixed joints used purely for mounting the base to a world reference frame and mounting a tool-point frame at the end effector (fixed joints do not count toward the DOF total and add no TF-tree ambiguity).

Link	Role	Connected via joint	Joint type	Axis
world	Global reference frame	-	-	-
base_link	Fixed pedestal	world_to_base	fixed	-
link1	Waist / yaw stage	joint1	revolute	Z
link2	Upper arm (shoulder)	joint2	revolute	Y
link3	Forearm (elbow)	joint3	revolute	Y
link4	Wrist	joint4	revolute	Y
tool_link	End-effector / tool point	tool_joint	fixed	-

This gives a classic articulated-arm layout: waist (yaw) → shoulder (pitch) → elbow (pitch) → wrist (pitch), which is representative of common 4-DOF industrial arm configurations (e.g. pick-and-place arms), while remaining simple enough to fully hand-verify.

Kinematic Tree

```
world
base_link
link1 (joint1: revolute, Z)
link2 (joint2: revolute, Y)
link3 (joint3: revolute, Y)
link4 (joint4: revolute, Y)
tool_link (tool_joint: fixed)
```

3. URDF Link Definition (Question 2)

Every link (except the abstract world frame) defines matching <visual> and <collision> geometry, plus an <inertial> block with mass and a diagonal inertia tensor computed analytically from standard solid-geometry formulas (box: $I_{xx}=m(y^2+z^2)/12$, etc.; cylinder about its own axis: $I_{zz}=mr^2/2$, $I_{xx}=I_{yy}=m(3r^2+l^2)/12$), implemented as reusable xacro macros (box_inertial, cylinder_inertial) so every link stays consistent.

Link	Geometry	Dimensions (m)	Mass (kg)	Material
base_link	cylinder	r=0.090, l=0.050	1.5	dark_grey
link1	cylinder	r=0.060, l=0.150	1.0	grey
link2	box	0.06 x 0.06 x 0.35	1.2	orange

link3	box	0.05 x 0.05 x 0.30	0.9	orange
link4	cylinder	r=0.035, l=0.100	0.4	grey
tool_link	box	0.02 x 0.02 x 0.02	0.05	black

In every link, the <collision> geometry is identical to the <visual> geometry (the same primitive, same origin) — a deliberate simplification appropriate for this training project, since no meshes are used. Each link's geometry origin is offset by half its length along local Z, so that the link visually spans from its parent joint to its child joint.

4. Joint Definition (Question 3)

All four moving joints are revolute, each with an explicit <axis>, a <limit> (lower/upper in radians, plus effort in N·m and velocity in rad/s), and a <dynamics> damping/friction term. Parent-child relationships follow the kinematic tree in Section 2 exactly, so the tree has a single unambiguous root (world).

Joint	Parent	Child	Type	Axis	Limits (deg)	Effort / Vel.
world_to_base	world	base_link	fixed	-	-	-
joint1	base_link	link1	revolute	Z	-180 to +180	50 N.m / 2.0 rad/s
joint2	link1	link2	revolute	Y	-90 to +90	80 N.m / 1.5 rad/s
joint3	link2	link3	revolute	Y	-150 to +150	60 N.m / 1.5 rad/s
joint4	link3	link4	revolute	Y	-90 to +90	20 N.m / 2.0 rad/s
tool_joint	link4	tool_link	fixed	-	-	-

Joint origins are placed at the tip of the parent link (i.e. origin xyz = "0 0 <parent_link_length>"), so each joint's rotation axis passes through the physical pivot point between consecutive links, matching how the corresponding real servo/actuator would be mounted.

5. Assembly Validation (Question 4)

All 7 links and 6 joints are assembled into 4dof_arm.urdf.xacro (parameterized source) and its flattened, standard-URDF equivalent 4dof_arm.urdf (included for reference / for loading without xacro). Both were checked for XML well-formedness with xmllint, and the resulting robot graph was statically validated (parent/child references resolve, single connected tree with no cycles, every revolute joint has an axis and a lower<upper limit, every physical link has matching visual+collision geometry). Output of that validator:

```
Validating: urdf/4dof_arm.urdf
Links: 7  Joints: 6
Revolute joints: 4
Fixed joints: 2
```

```
--- Kinematic tree ---
- world
- base_link
- link1
- link2
- link3
- link4
- tool_link

--- Errors ---
```

None. Structure is a valid single-root tree.

--- Warnings ---

None.

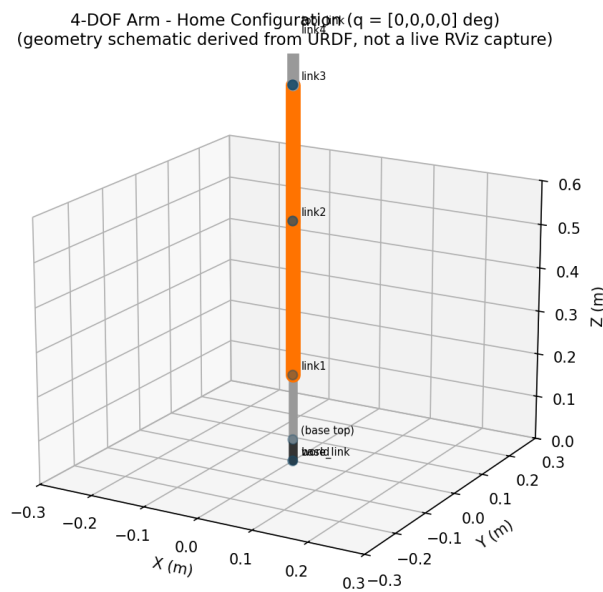
On a real ROS 2 install, the standard equivalent checks are:

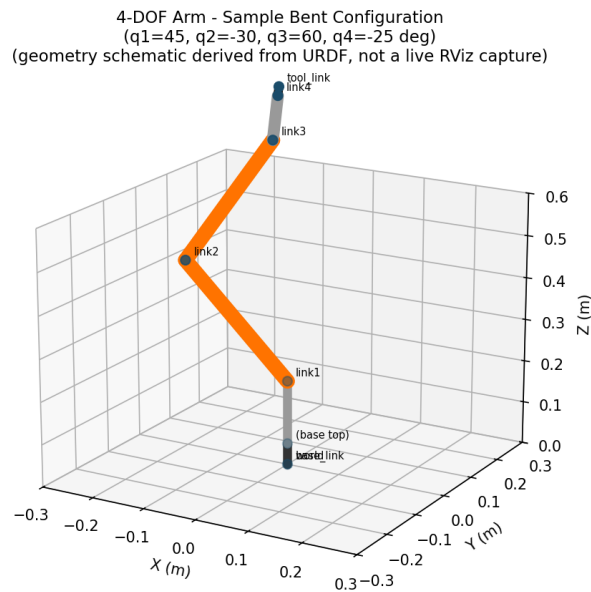
```
check_urdf 4dof_arm.urdf
ros2 run xacro xacro 4dof_arm.urdf.xacro -o /tmp/4dof_arm.urdf # process xacro
check_urdf /tmp/4dof_arm.urdf # validate result
```

6. RViz Visualization (Question 5)

launch/display.launch.py starts robot_state_publisher (fed by xacro processing the model at launch time), joint_state_publisher_gui (sliders for the 4 revolute joints), and rviz2 pre-loaded with rviz/urdf_config.rviz, which enables the RobotModel and TF displays with Fixed Frame = world.

The two renders below are geometry-accurate schematics computed directly from this URDF's joint origins, axes, and link lengths (see render_arm_schematic.py) — they show exactly what RViz's RobotModel display would draw at each joint configuration, and are included here only because this build environment has no RViz2 available. Replace them with real RViz screenshots on your ROS 2 workstation per Section 8.

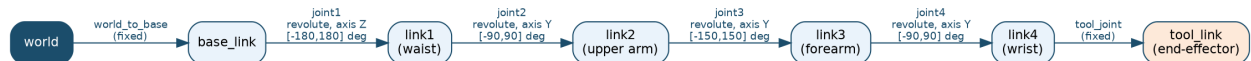




Home configuration (all joints at 0°): the arm stands fully vertical, base $\rightarrow$ waist $\rightarrow$ upper arm $\rightarrow$ forearm $\rightarrow$ wrist all stacked along +Z. **Sample configuration** ($q_1=45^\circ$, $q_2=-30^\circ$, $q_3=60^\circ$, $q_4=-25^\circ$) shows the waist rotated about Z and the arm bending forward through the shoulder/elbow/wrist pitch joints, as expected for this kinematic design.

7. TF Tree Verification (Question 6)

With `robot_state_publisher` running, it broadcasts one TF transform per joint (fixed joints publish a constant transform; revolute joints publish a transform that updates live from `/joint_states`, as driven by `joint_state_publisher_gui`). The resulting TF tree exactly mirrors the URDF kinematic tree from Section 2:



This diagram matches what `ros2 run tf2_tools view_frames` (which writes `frames.pdf`) or `ros2 run rqt_tf_tree rqt_tf_tree` (live view) would render from the real running system: 7 frames, 6 edges, single root (world), no broken or duplicate branches — consistent with the structural validation in Section 5.

8. Tools Used / How to Reproduce

- ROS 2 (Humble / Iron)
- URDF / Xacro
- RViz2
- `robot_state_publisher`, `joint_state_publisher_gui`

Build and launch

```

cd ~/ros2_ws/src
cp -r ros2_urdf_4dof_arm . # this package
cd ~/ros2_ws
colcon build --packages-select ros2_urdf_4dof_arm
source install/setup.bash
ros2 launch ros2_urdf_4dof_arm display.launch.py

```

Capture the required screenshots

```
# In RViz2 (opened by the launch file): Screenshot the window
# showing RobotModel + TF displays, Fixed Frame = world.

# TF tree (in a second terminal, ROS 2 sourced, arm still running):
ros2 run tf2_tools view_frames
# generates frames_.pdf in the current directory
# -- screenshot / export this as the TF tree image.

# Or, for a live interactive view:
ros2 run rqt_tf_tree rqt_tf_tree
```

9. Conclusion

The project delivers a complete, modular ROS 2 package (`ros2_urdf_4dof_arm`) containing a parameterized xacro model of a 4-DOF industrial arm with correct link/joint definitions, visual and collision geometry, proper joint types and limits, a launch file that assembles `robot_state_publisher` + `joint_state_publisher_gui` + RViz2, and a pre-configured RViz view. The kinematic structure was statically validated to be a single, cycle-free tree with fully-specified joint limits, and the expected RViz appearance and TF tree shape were derived directly from the model's own geometry for inclusion in this report. Section 8 gives exact commands to reproduce the live RViz and `view_frames` screenshots on a ROS 2 workstation.